

DevOps Halloween Brown Bag

Bad Packages

Oct 2025

By Anthony McKale



Internal use only
Image generated by AI

DevOps Halloween Brown Bag & Panel

Bad Packages

Oct 2025
By Anthony McKale

Internal use only
Image generated by AI

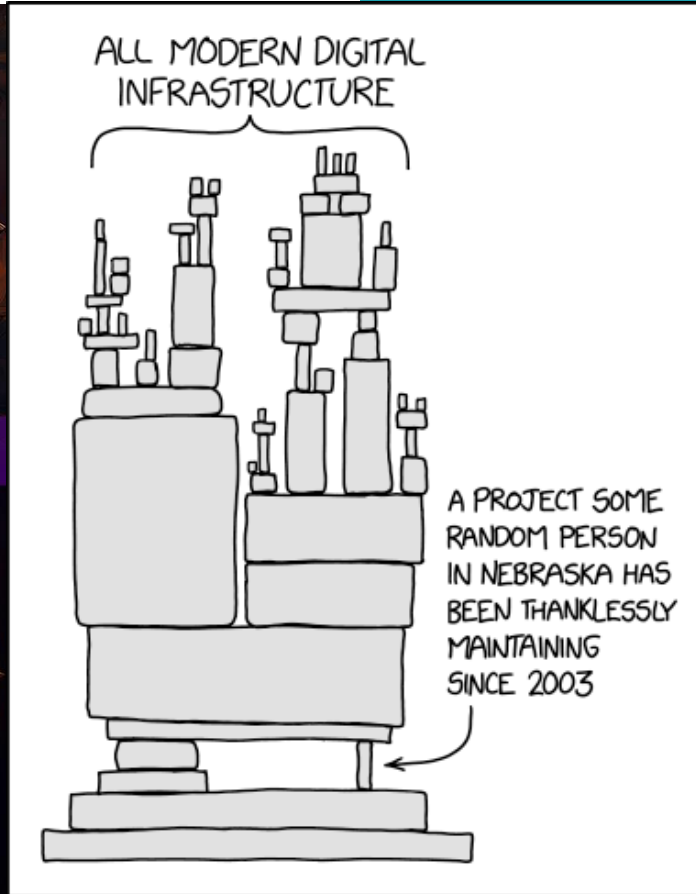
Package of Contents

- Packages 101
- Package Security 101
- Package Supply Chain Gone Wrong
- Protestware Example
- Crypto Spyware Example
- Worm Spyware Example
- Q&A



Packages 101

[xkcd 2347 : Dependency](#)



SIMPLE: Packages 101 (npm example)

npm install (think mvn build)

package.json

identify the
dependencies listed

Fetch

npm fetches the
required packages
from npm registry

Build Tree

Builds dependency
tree

Ensure compatibility
and resolves conflicts

Installation

Install packages into
node_modules folder

DETAILS: Packages 101 (npm example)

npm install (think mvn build)

package.json

identify the dependencies listed

Identify the devDependencies listed

Detect versions considering version ranges (e.g., ^, ~, or =).

Fetch

npm fetches the required packages from npm registry

When already installed, may use the cached version

Build Tree

Builds dependency tree

Ensure compatibility and resolves conflicts

Deduplication of shared dependencies

Installation

Install packages into node_modules folder

Symlinks for binaries

Post-Install Scripts:

Execute post install steps in package.json

Execute post-install scripts of the installed packages

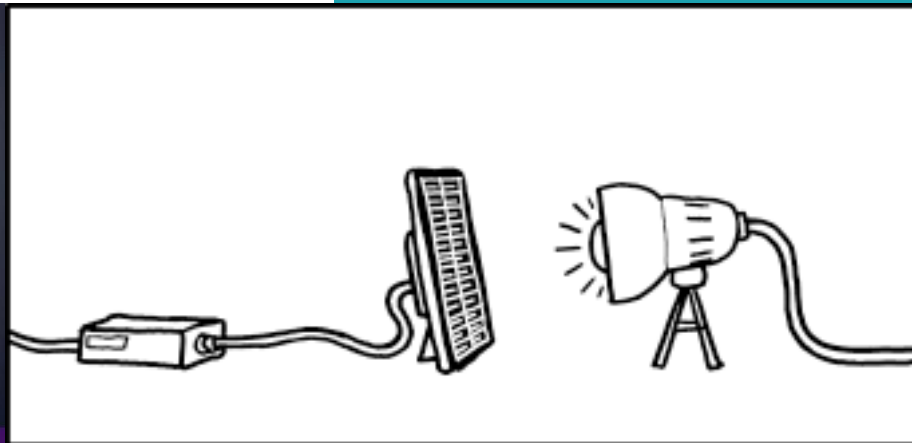
Package Security 101

[xkcd: Air Gap](#)



Internal use only

Image generated by AI



ENERGY TIP: INCREASE THE SECURITY OF YOUR HOME POWER SUPPLY BY INSTALLING AN AIR GAP.

SIMPLE: Package Security 101

Software Comp Analysis (SCA)

Run SCA to analyse the dependency tree
(e.g. npm audit)

Review results: CVEs

Remediate Issues

Automate in CICD and security Gate

Static App Security Testing (SAST)

Run SAST to analyse Code
(e.g. eslint)

Review results:
insecure coding patterns and
hardcoded secrets

Remediate Issues

Automate in CICD and security Gate

Dynamic App Security Testing (DAST)

Run DAST to analyse your deployed app
(e.g. OWASP ZAP)

Review results:
XSS,SQL Injection, Insecure or
Broken authentication or sessions

Remediate Issues

Automate in CICD and security Gate

DETAILS: Package Security 101

Normal sane steps, are you doing all 4...

Software Comp Analysis (SCA)

NPM install all dependencies *
(CLUE)

Run SCA to analyse the
dependency tree
(e.g. npm audit)

Review results: CVEs

Remediate Issues

Automate in CI/CD and security
Gate

Static App Security Testing (SAST)

No install required

Run SAST to analyse Code
(e.g. eslint)

Review results:
insecure coding patterns and
hardcoded secrets

Remediate Issues

Automate in CI/CD and security
Gate

Dynamic App Security Testing (DAST)

Deploy App * (CLUE)

Run DAST to analyse your
deployed app
(e.g. OWASP ZAP)

Review results:
XSS, SQL Injection, Insecure or
Broken authentication or sessions

Remediate Issues

Automate in CI/CD and security
Gate

Harden or Sandbox Boxes

**Harden NPM: Disable post
scripts**

**Harden NPM: Wrap in safe-chain
or similar**

**Harden custom registry with
minimum ages to prevent zero-
day**

**Run npm in low trust sandbox in
CI/CD and IDE**

Where it goes Pete Tong



Point to the attack Vectors

If I was a bad man, where would I target ?

package.json

identify the dependencies listed

identify the devDependencies listed

Detect versions considering version ranges (e.g., ^, ~, or =).

Fetch

npm fetches the required packages from npm registry

When already installed, may use the cached version

Build Tree

Builds dependency tree

Ensure compatibility and resolves conflicts

Deduplication of shared dependencies

Installation

Install packages into node_modules folder

Symlinks for binaries

Post-Install Scripts:

Execute post install steps in package.json

Execute post-install scripts of the installed packages

Point to the attack Vectors

LOCAL AND CICD... and of course in the final Product

package.json

identify the dependencies listed

identify the devDependencies listed

Detect versions considering version ranges (e.g., ^, ~, or =).

Fetch

npm fetches the required packages from npm registry

When already installed, may use the cached version

Build Tree

Builds dependency tree

Ensure compatibility and resolves conflicts

Deduplication of shared dependencies

Installation

Install packages into node_modules folder

Symlinks for binaries

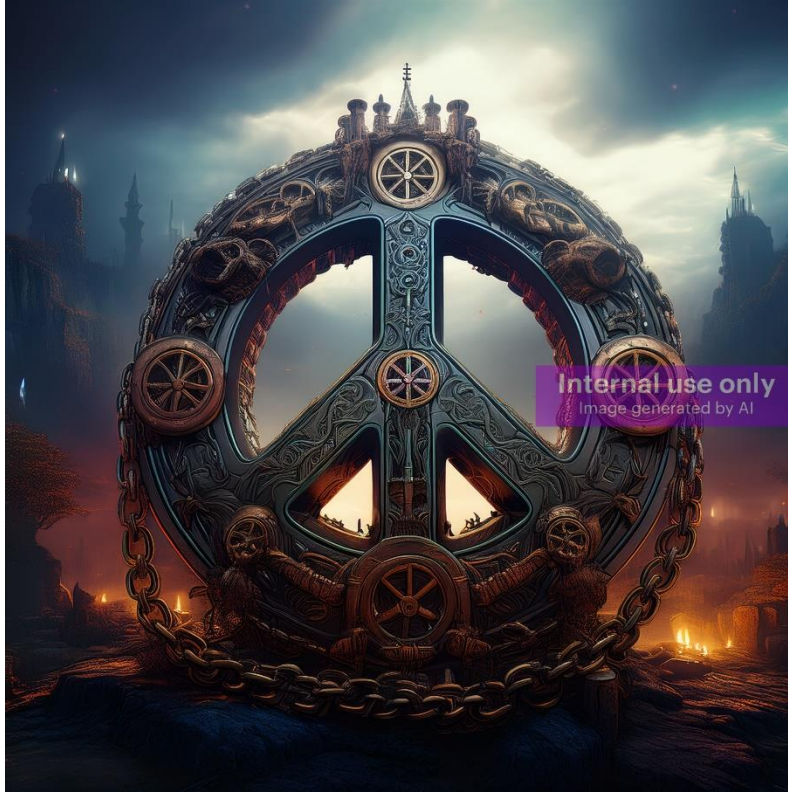
Post-Install Scripts:

Execute post install steps in package.json

LOCAL: Execute post-install scripts of the installed packages

CICD: Execute post-install scripts of the installed packages

Protestware: Peace not War



<https://en.wikipedia.org/wiki/Peacenotwar>

Protestware: Peace not War

- Between 7 March and 8 March 2022, Maintainer **Brandon Miller** updated the **npm node-ipc** package
- The update caused the package to create a file on the desktops of all installed machines
- The file was named: **WITH-LOVE-FROM-AMERICA.txt**
- The file contained anti-war messages
- But can anyone guess what else it did?

<https://en.wikipedia.org/wiki/Peacenotwar>

Protestware: Peace not War

- gzip version uploaded to npm which didn't match the source... weird
- Post Install did GeoIP checks... weird
- Overwrote all Russian or Belarusian machines with heart emojis...
- Fun fact, NGOs collecting war crime images most affected...
- Fun Fact 2: node-ipc was a dependency of common packages such as: colors, vue.js, and unity

Famous NPM Package Deletes Files To Protest Ukraine War - Slashdot
<https://en.wikipedia.org/wiki/Peacenotwar>

ATTACK VECTOR

CICD AND DEVELOPERS

package.json

identify the dependencies listed

Identify the devDependencies listed

Detect versions considering version ranges (e.g., ^, ~, or =).

Fetch

npm fetches the required packages from npm registry

When already installed, may use the cached version

Build Tree

Builds dependency tree

Ensure compatibility and resolves conflicts

Deduplication of shared dependencies

Installation

Install packages into node_modules folder

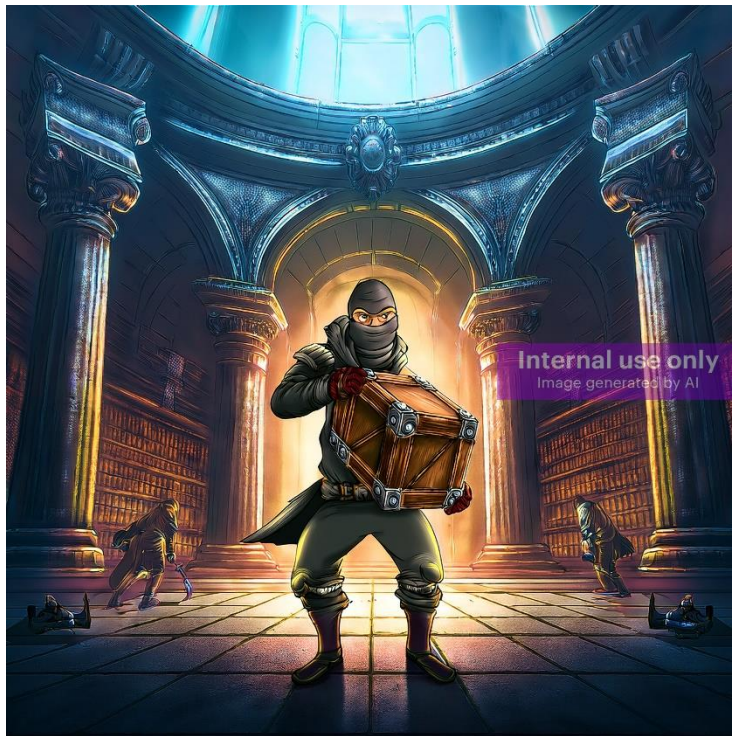
Symlinks for binaries

Post-Install Scripts:

Execute post install steps in package.json

Execute post-install scripts of the installed packages

Crypto Spyware: Debug Chalk



<https://www.aikido.dev/blog/npm-debug-and-chalk-packages-compromised>

Crypto Spyware: Debug Chalk

- Chalk and Debug Maintainers accounts taken by phishing attack (and others)
- 8th of September all maintainer packages compromised
- Installed loggers into deployed applications to steal crypto wallet credentials and redirect transfers
- 13th September compromised packages replaced and removed
[NVD - CVE-2025-59144](#)

<https://www.aikido.dev/blog/npm-debug-and-chalk-packages-compromised>

ATTACK VECTOR

BUILT ASSETS

package.json

identify the dependencies listed

Identify the devDependencies listed

Detect versions considering version ranges (e.g., ^, ~, or =).

Fetch

npm fetches the required packages from npm registry

When already installed, may use the cached version

Build Tree

Builds dependency tree

Ensure compatibility and resolves conflicts

Deduplication of shared dependencies

Installation

Install packages into node_modules folder

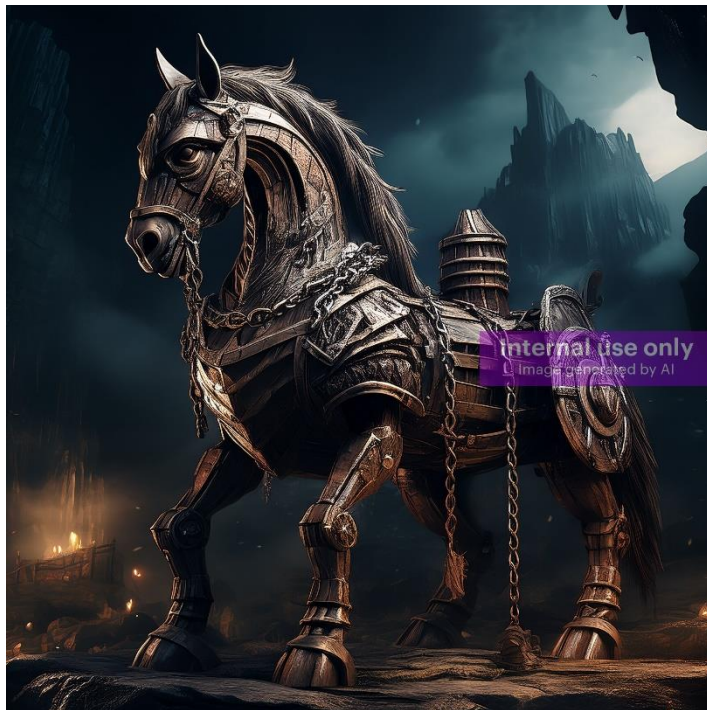
Symlinks for binaries

Post-Install Scripts:

Execute post install steps in package.json

Execute post-install scripts of the installed packages

Worm Spyware: Shai-Hulud



Shai-Hulud malware campaign dubbed 'the largest and most dangerous npm supply-chain compromise in history' — 'hundreds' of JavaScript packages affected | Tom's Hardware

Worm Spyware: Shai-Hulud

- 15th of September packages compromised with Shai-Hulud Worm start to appear
- Once installed sends npm/git/cloud api token home (that's for developer and CI/CD machines)
- Uses NPM token to infect any maintained packages
- As of 3rd October, Worm still active, with hundreds of popular packages and thousands of developer's machines compromised
"Shai-Hulud" Worm Compromises npm Ecosystem in Supply Chain Attack (Updated September 23)

Shai-Hulud malware campaign dubbed 'the largest and most dangerous npm supply-chain compromise in history' — 'hundreds' of JavaScript packages affected | Tom's Hardware

ATTACK VECTOR

CICD & DEVELOPERS, AND ALL THEIR SECRETS!

package.json

identify the dependencies listed

Identify the devDependencies listed

Detect versions considering version ranges (e.g., ^, ~, or =).

Fetch

npm fetches the required packages from npm registry

When already installed, may use the cached version

Build Tree

Builds dependency tree

Ensure compatibility and resolves conflicts

Deduplication of shared dependencies

Installation

Install packages into node_modules folder

Symlinks for binaries

Post-Install Scripts:

Execute post install steps in package.json

Execute post-install scripts of the installed packages

Worm Spyware: Shai-Hulud

- Follow guidance

CISA

Widespread Supply Chain Compromise Impacting npm Ecosystem | CISA

Palo Alto

"Shai-Hulud" Worm Compromises npm Ecosystem in Supply Chain Attack (Updated September 23)

- "Shai-Hulud" worm is a simplistic and obvious attack, likely from non-state actors

Expect more sophisticated threats in local package management systems!

Shai-Hulud malware campaign dubbed 'the largest and most dangerous npm supply-chain compromise in history' — 'hundreds' of JavaScript packages affected | Tom's Hardware

Q&A

